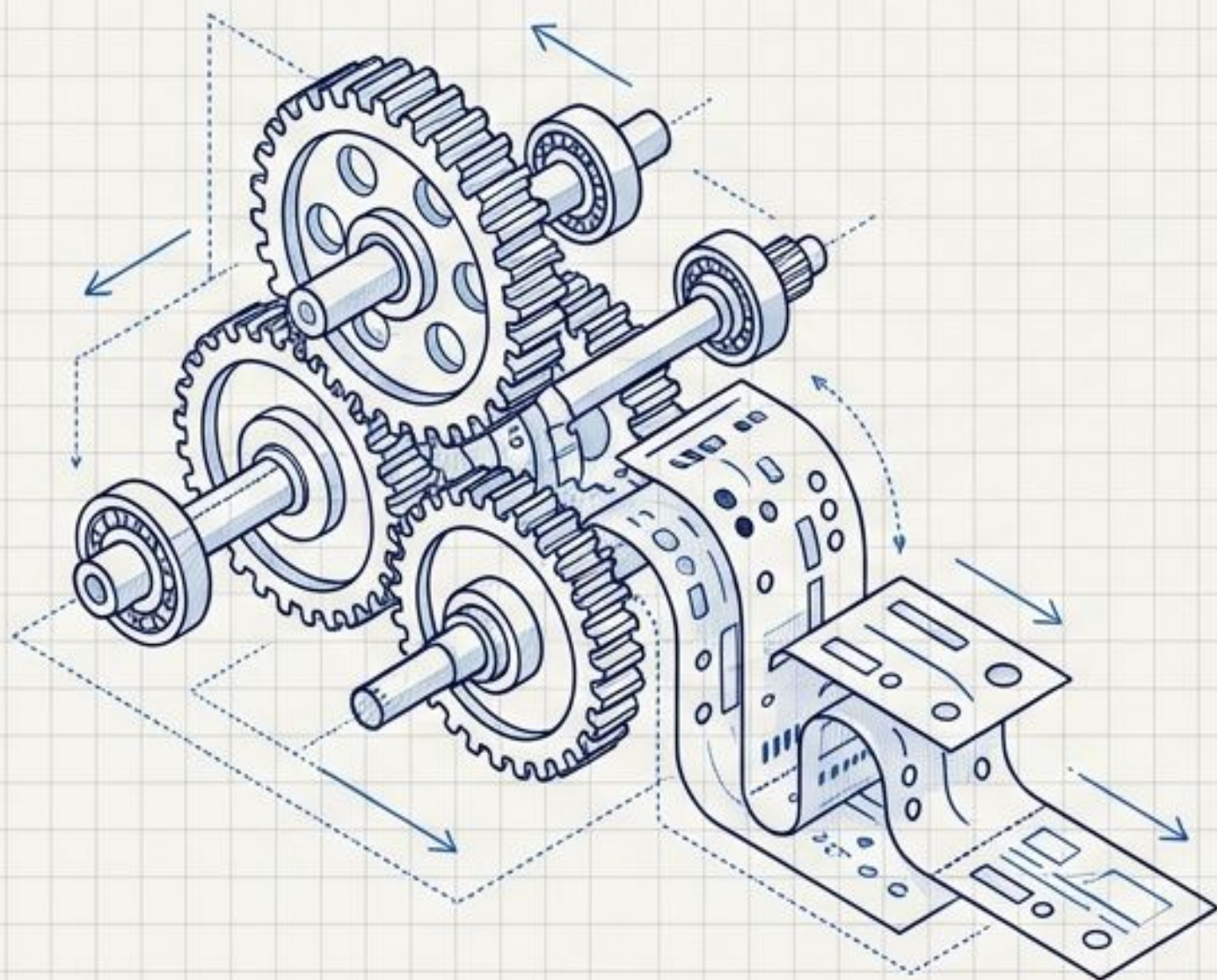


# Engenharia de Software com Agentes de IA

Orquestrando o desenvolvimento end-to-end com o Cursor IDE.



## Agentes

[.cursor/agents/](#)

Papéis altamente especializados. Representam a QUEM delegamos tarefas específicas no fluxo de desenvolvimento (ex: `Business Analyst`, `Backend Developer`). Eles executam o trabalho ativo.

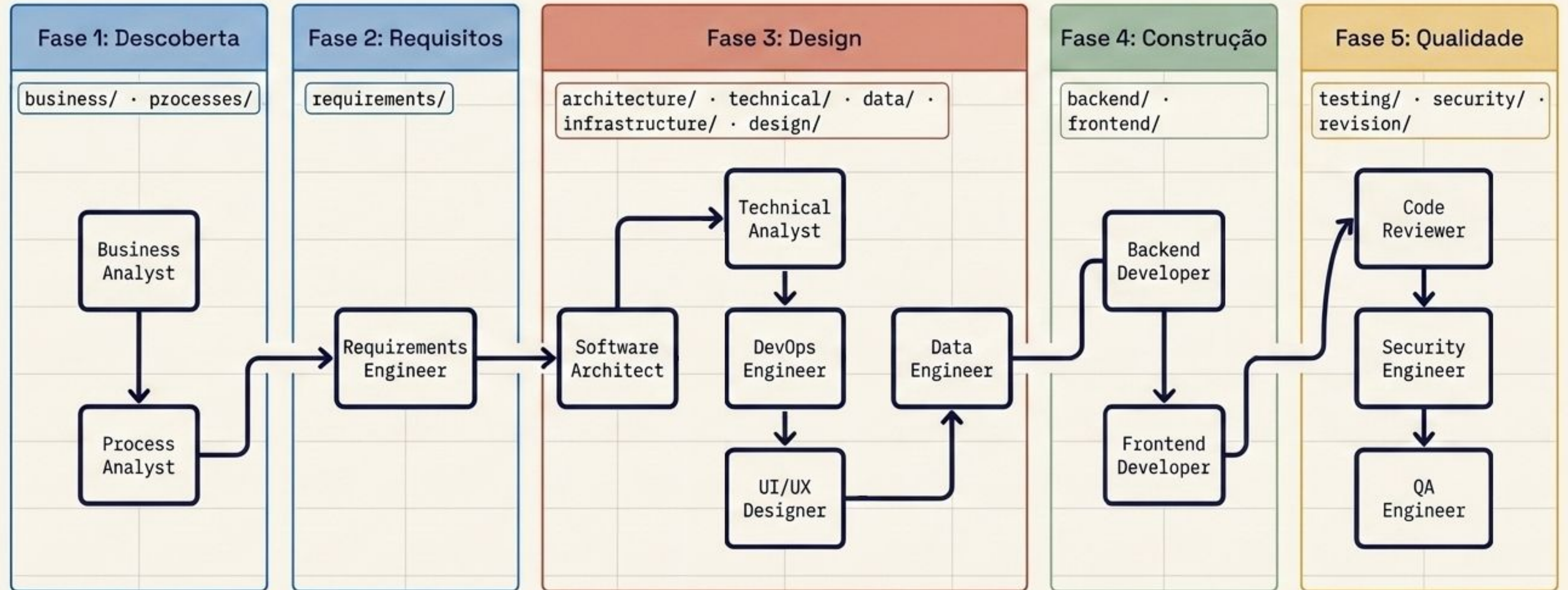


## Skills

[.cursor/skills/](#)

Instruções de base, regras e referências de contexto. Representam o COMO os agentes executam o trabalho, garantindo padronização, rastreabilidade e qualidade arquitetural.

# O Pipeline de Agentes: 5 Fases, 14 Especialistas



Processo Transversal: Codebase Documenter (Acompanha todo o ciclo)

# Business Analyst

## FASE 1: DESCOBERTA



### 0 Papel

- Entende profundamente as necessidades do cliente e dos stakeholders.
- Define a visão do produto, os objetivos estratégicos e as métricas de sucesso.
- Executa a gap analysis (comparando o estado atual com o estado desejado).

### Gatilho de Ação

0 primeiro do pipeline. Acionado no início absoluto de um novo projeto ou na redefinição de um escopo.

### Skill

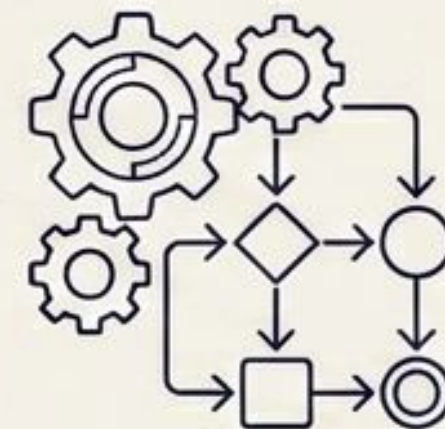
`</> business-analysis`

### Artefatos Gerados

```
business/  
├── product-vision.md  
├── stakeholder-matrix.md  
├── business-requirements.md  
└── gap-analysis.md
```

# Process Analyst

## FASE 1: DESCOBERTA



### 0 Papel

- Mapeia os processos de negócio atuais utilizando notação BPMN.
- Identifica gargalos operacionais e processos críticos.
- Propõe oportunidades claras de melhoria e automação tecnológica.

### Gatilho de Ação

Acionado imediatamente após a conclusão da análise de negócios.

[Dependência direta: Business Analyst]

### Skill

</> process-mapping

### Artefatos Gerados

```
processes/  
├── process-map.md  
├── critical-processes-matrix.md  
├── improvement-opportunities.md  
└── [Diagramas BPMN]
```

# Requirements Engineer

## FASE 2: REQUISITOS



## 0 Papel

- Produz o documento SRS (Software Requirements Specification), detalhando RFs e RNFs.
- Prioriza o backlog utilizando frameworks (ex: MoSCoW) e gerencia a matriz de rastreabilidade.
- Decompõe grandes demandas em User Stories atômicas com critérios de aceite em formato Gherkin.

## Gatilho de Ação

Acionado após a definição e aprovação dos processos.

[Depende de: Business e Process Analysts]

## Skills (Múltiplas)

```
</> requirements-spec
```

(Focado na criação do SRS e estruturação do Backlog)

```
</> user-story-decomposition
```

(Focado em preparar histórias no padrão "Ready for Dev")

## Artefatos Gerados

```
requirements/  
├── srs.md  
├── prioritized-backlog.md  
└── user-stories-ready-for-dev.md
```

# Software Architect

FASE 3: DESIGN

🔑 NÓ CENTRAL

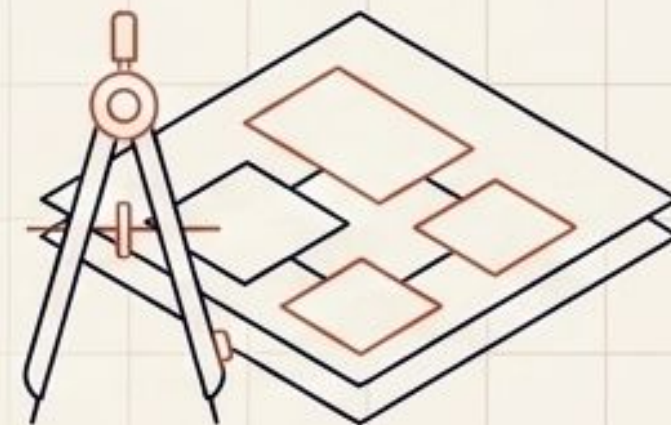
## O Papel

- Define o 'Grande Desenho': escolhe entre monólito vs. microsserviços, define a stack de tecnologia e padrões principais.
- Especifica atributos de qualidade (performance, escalabilidade) e APIs principais.
- Registra decisões estruturais através de ADRs (Architecture Decision Records), detalhando alternativas e trade-offs.

## Gatilho de Ação

Acionado assim que os requisitos estão especificados.

**Sua conclusão habilita o trabalho em paralelo massivo na fase de design.**



## Skill

`</> architecture-design`

## Artefatos Gerados

```
architecture/  
├── architecture-document.md  
├── technology-stack.md  
├── c4-diagrams/  
└── adrs/
```

# Technical Analyst

FASE 3: DESIGN

[TRILHA PARALELA 1/3]



## O Papel

- Escova os bits da arquitetura, detalhando especificações técnicas por componente.
- Define contratos rigorosos de API (padrão OpenAPI/Swagger) e modelos lógicos de dados (ERD).
- Documenta minúcias de integrações, padrão BFF (Backend For Frontend), políticas de CORS e casos de uso técnicos.

## Gatilho de Ação

Roda simultaneamente com DevOps e UI/UX.

[Depende exclusivamente do Software Architect]

## Skill

`</> technical-spec`

## Artefatos Gerados

```
└─ technical/
    └─ ── technical-specifications.md
    └─ ── api-contracts/
    └─ ── data-models/
    └─ ── integration-specs.md
```

# Technical Analyst

FASE 3: DESIGN

[TRILHA PARALELA 1/3]

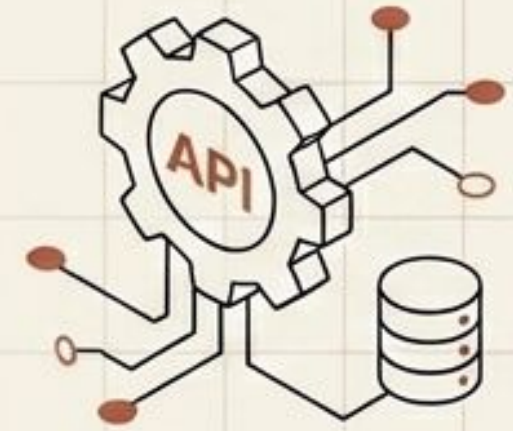
## Dica bônus

- Adicionalmente pode-se pedir ao agente para refinar, detalhar e descrever melhor as atividades técnicas das U.S. em user-stories-ready-for-dev.md. nos grupos Backend, Frontend, Data e Test.

## Gatilho de Ação

Roda simultaneamente com DevOps e UI/UX.

[Depende exclusivamente do Software Architect]



## Skill

```
</> technical-spec
```

## Artefatos Gerados

```
└─ technical/
   └─ ── technical-specifications.md
   └─ ── └─ api-contracts/
   └─ ── └─ data-models/
   └─ ── └─ ── integration-specs.md
```

# DevOps Engineer

FASE 3: DESIGN

[TRILHA PARALELA 2/3]

## 0 Papel

- Prepara toda a infraestrutura base e os fluxos de entrega contínua.
- Cria os Dockerfiles, manifests Kubernetes e docker-compose para ambientes dev e prod.
- Configura os pipelines de CI/CD para dev, staging e prod.
- Garante higiene de repositório mantendo o .env.example livre de secrets.

## Gatilho de Ação

Roda simultaneamente com o Technical Analyst e UI/UX.

[Recebe insumos do Software Architect]

## Skill

</> devops-infra

## Artefatos Gerados

```
└─ infrastructure/
   └─ dockerfiles/
      └─ docker-compose.*.yaml
   └─ kubernetes/
   └─ ci-cd/
   └─ README.md (e .env.example)
```





## 0 Papel

- Desenha a interface final: do mapa de páginas e wireframes aos mockups de alta fidelidade.
- Cria e padroniza o Design System.
- Garante acessibilidade (normas WCAG), abordagem mobile-first e projeta estados complexos de interface (loading, erro, sucesso).

## Gatilho de Ação

Roda simultaneamente com Arquitetura e Specs Técnicas.

[Orientado pelas User Stories do Requirements Engineer]

## Skills (Múltiplas)

### `</> uiux-design`

(Foco em fluxos de jornada, componentização e regras de acessibilidade)

### `</> interface-design`

(O craft visual e estético de painéis e aplicações)

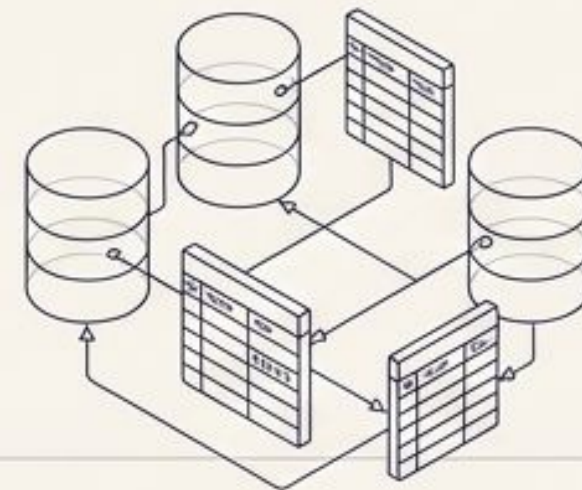
## Artefatos Gerados

```
design/  
├── design-system.md  
├── component-specs.md  
└── wireframes/ & mockups/
```

# Data Engineer

FASE 3: DESIGN

[FIM DO CICLO DE DESIGN]



## O Papel

- Modela entidades, relacionamentos e regras absolutas de persistência de dados.
- Produz scripts estruturais (DDL) e arquitetura de migrações em alinhamento com as ADRs.
- Trata no nível de dados as políticas de autenticação, tokens e expiração.

## Gatilho de Ação

Aguarda a definição da Arquitetura, Especificações Técnicas e Infraestrutura base. Prepara o terreno exato para o código.

## Skill

`</> data-engineering`

## Artefatos Gerados

```
data/  
├── modelo-dados.md  
├── glossário.md  
└── [Scripts de esquema / migrações]
```



**Próximo Passo:** Entrega o bastão diretamente para o Backend Developer iniciar a implementação da persistência.

# Backend Developer

FASE 4: CONSTRUÇÃO

## 0 Papel

- Implementa o 'motor': APIs REST/GraphQL, regras rígidas de lógica de negócio e persistência de dados.
- Desenvolve integrações externas e sistemas de autenticação (baseados em JWT).
- Aplica padrões de projeto avançados (ex: padrão Facade para composição de serviços) e trata o CORS.

## Gatilho de Ação

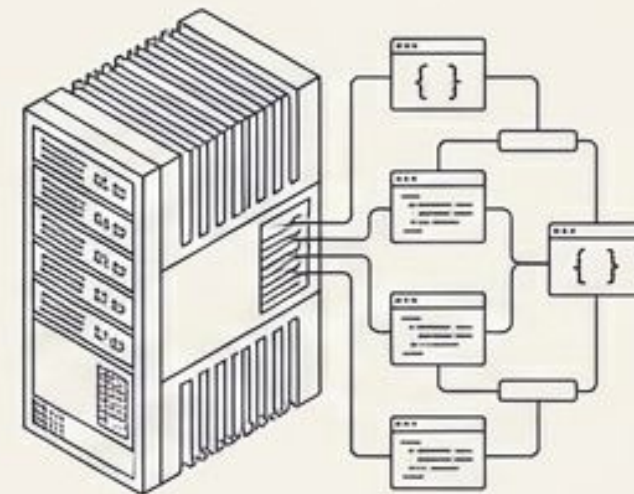
Recebe todos os insumos de Technical, Data, DevOps e consome ativamente o backlog de User Stories.

## Skill

`</>` backend-dev

## Artefatos Gerados

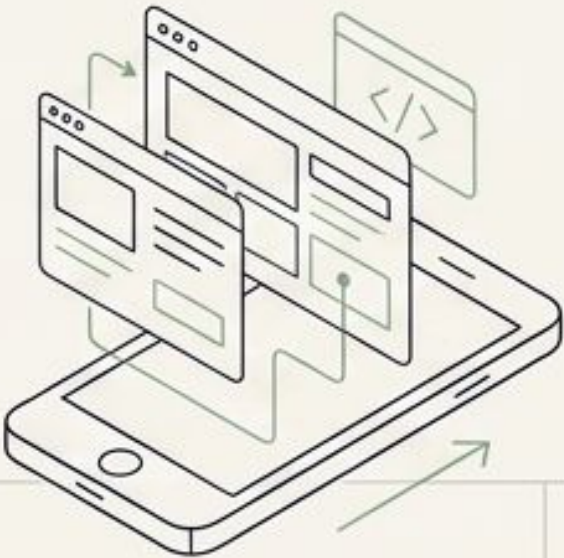
📁 backend/  
├── 💻 [Código-fonte]  
├── 🧪 [Testes unitários / de integração]  
└── 📄 [Documentação de API OpenAPI]



⚡ **Gatilho de Cascata:** O Backend aciona imediatamente o QA Engineer, Security e Documenter a cada nova entrega.

# Frontend Developer

FASE 4: CONSTRUÇÃO



## O Papel

- Constrói a interface pixel-perfect seguindo o Design System.
- Gerencia estados de aplicação, roteamento e integra o client com as APIs do backend.
- Otimiza acessibilidade, Core Web Vitals (performance) e gerencia estratégias de renderização (SSG/SSR).

Regra de Ouro: Zero persistência de negócio local (dados transacionados apenas via backend).

## Gatilho de Ação

Atua de forma alinhada à Construção.

[Depende do UI/UX Designer para estética e do Technical Analyst para contratos de API]

## Skill

`</> frontend-dev`

## Artefatos Gerados

 frontend/  
├─  [Componentes isolados]  
├─  [Páginas de aplicação]  
└─  [Testes de componente]

# Code Reviewer

FASE 5: QUALIDADE

## 0 Papel

- Inspecciona o código base através de três lentes críticas autônomas.
- Consolida todos os achados em uma "lista de afazeres" e distribui as responsabilidades de correção para as respectivas equipes de dev, infra ou design.

## Gatilho de Ação

Acionado estritamente sob demanda (Eventos pré-merge, revisão isolada de um diff ou análise de um módulo completo recém-concluído).



## Skill

```
</> code-review
```

## As 3 Lentes & Artefatos

```
revision/
├── 🔍 Lente 1: Impacto → 📄 regressao-impacto.md
├── 🔍 Lente 2: Blindagem → 📄 seguranca.md
├── 🔍 Lente 3: Manutenibilidade → 📄 clean-code.md
└── 📋 Consolidação Final → 📄 RELATORIO-UNIFICADO.md
```

# Security Engineer

FASE 5: QUALIDADE



## 👤 0 Papel

- Auditoria de segurança de aplicações ativas (Práticas de AppSec).
- Valida a aderência ao OWASP Top 10, a integridade da autenticação, e a correta aplicação de tokens (ex: JWT) e hashing criptográfico de senhas.
- Rastreia vazamentos: garante absolutamente que nenhum secret ou chave privada escapou para o código-fonte.

## ⚡ Gatilho de Ação

Condicionado às entregas primárias de Frontend e Backend estarem concluídas.

## ≡ Skill

```
</> security-audit
```

## 📁 Artefatos Gerados

```
security/  
├── [Relatório de vulnerabilidades]  
└── [Recomendações técnicas de correção]
```

# QA Engineer

FASE 5: QUALIDADE



## 0 Papel

- Garante a robustez e evita regressões de sistema.
- Gera e executa suítes de testes cumulativas, combinando testes de novas features com a bateria de regressão.
- Emite relatórios unificados de falhas e análise de cobertura de código.

## Gatilho de Ação

Processo contínuo. Rodado cirurgicamente logo após a implementação de cada User Story.

## Skill

</> qa-testing

## Aparato de Teste (Execução em container isolado Node+Playwright)

| Nível      | Ferramenta | Pasta                |
|------------|------------|----------------------|
| Unitário   | Jest       | testing/unit/        |
| Integração | Jest + MSW | testing/integration/ |
| E2E        | Playwright | testing/e2e/         |

# Codebase Documenter

PROCESSO TRANSVERSAL / ACOMPANHA TODO O FLUXO



## O Papel

- Garante a manutenibilidade futura do ecossistema.
- Insere documentação invisível (inline, comentários inteligentes, docstrings nas funções).
- Alinha rigorosamente a documentação interna em Markdown com os contratos de API públicos (OpenAPI).

## Gatilho de Ação

Acompanhamento contínuo. Acionado de forma automática logo após qualquer entrega estável de dev, infra, design ou security.

## Skill

```
</> codebase-documenter
```

## Artefatos Gerados (Geração Capilarizada)

📁 [Qualquer diretório alvo]

- ├── 📄 README.md (gerado em todas as pastas raiz)
- └── 📁 documentation/ (ex: infrastructure/documentation/)

# Matriz de Especialização: Agentes & Skills

| AGENTE (Quem executa) | SKILL (Como executa)                               |
|-----------------------|----------------------------------------------------|
| Business Analyst      | </> business-analysis                              |
| Process Analyst       | </> process-mapping                                |
| Requirements Engineer | </> requirements-spec </> user-story-decomposition |
| Software Architect    | </> architecture-design                            |
| Technical Analyst     | </> technical-spec                                 |
| Data Engineer         | </> data-engineering                               |
| DevOps Engineer       | </> devops-infra                                   |
| UI/UX Designer        | </> uiux-design </> interface-design               |
| Backend Developer     | </> backend-dev                                    |
| Frontend Developer    | </> frontend-dev                                   |
| QA Engineer           | </> qa-testing                                     |
| Security Engineer     | </> security-audit                                 |
| Code Reviewer         | </> code-review                                    |
| Codebase Documenter   | </> codebase-documenter                            |

**Agente = QUEM executa no fluxo | Skill = COMO executar com qualidade e rastreabilidade.**